SWRL

语义网规则语言 · 规范速查

SWRL — OVERVIEW · OWL MEETS HORN RULES

01 总览 · OWL 缺的那一块

PART 1 · OVERVIEW

SWRL = OWL + Horn 规则:本体管分类与层级,规则管跨类的 if-then;代价是推理可能不可判定,所以支持度是选型关键。

OWL 底座	本体说「是什么」· 描述逻辑	分类与层级
规则层	body → head · Horn 子句	合取蕴含
构件	原子七类 · swrlb: 内置谓词	变量须安全
代价 · 生态	可判定性换表达力	推理机子集支持

主线:规则 = 原子的合取(前件)蕴含原子的合取(后件),变量全称量化。

四层各管一摊:底座给词汇,规则给推断,构件给写法,生态给边界。

本卡结构 = 本图:概览 → 构件 → 内置与代价 → 生态与实操。

02 定位 · OWL 缺的那一块

PART 1 · POSITION

OWL 描述「世界长什么样」,是名词性的公理;SWRL 描述「从已知能推出什么」,是动词性的规则。OWL 2 属性链补了一点,但带变量比较、函数计算的任意条件规则,仍然非 SWRL 不可。

层	管什么	一句例子
RDF	事实(三元组)	阿司匹林片 · 有适应症 · 解热镇痛
RDFS	类与属性的层级	处方药 ⊑ 药品
OWL	逻辑公理(交并补、基数、限制)	药品 ⊑ 至少 1 个 适应症
SWRL	条件规则(如果…则…)	有适应症 ∧ 该适应症属禁忌 → 需警示

STATUS · 先看清身份

SWRL 全称 **Semantic Web Rule Language**,2004-05-21 以 **W3C Member Submission**(会员提案)发布——提交方是加拿大国家研究委员会、Network Inference、斯坦福大学。**它不是 W3C Recommendation**,W3C 明说「发布不代表背书」。它把 OWL DL/Lite 与 Datalog RuleML 揉在一起:拿 OWL 的词汇当砖,按规则语言的形状盖房。

03 规则长什么样 · body → head

PART 1 · RULE ANATOMY

一条 SWRL 规则就是一次「如果…那么…」的蕴涵(implication):左半是前提(body / antecedent),右半是结论(head / consequent)。两边各自是若干原子的合取(conjunction),全都成立才推得出。

```
# 人读形式:parent(?x,?y) ∧ brother(?y,?z) ⇒ uncle(?x,?z)

# 规范的抽象语法:Implies( Antecedent(…) Consequent(…) )
Implies( Antecedent( :药品(?d) :有适应症(?d,?i) :禁忌症(?i) )
         Consequent( :需警示(?d) ) )
```

规范细节

两个边界值:**空前提**视为恒真(true),**空结论**视为恒假(false)。变量全称量化,作用域限于单条规则——同名的 `?d` 出了这条规则就不再相干。

XML 具体语法 · ruleml:imp 一条规则的真身

```
<ruleml:imp>
  <ruleml:_rlab ruleml:href="http://example.org/#r1"/>
  <ruleml:_body>
    <swrlx:classAtom> … </swrlx:classAtom>   # 每个原子一个元素
    <swrlx:individualPropertyAtom swrlx:property="…"> …
  </ruleml:_body>
  <ruleml:_head> … </ruleml:_head>
</ruleml:imp>
```

与 Datalog 的关系

SWRL 是 Horn 子句形状,与 Datalog 同构——但函数、常量全部换成 OWL 词汇,推理机也换成 DL 推理机+规则前端的组合。

SWRL

语义网规则语言 · 规范速查

SWRL — ATOMS, VARIABLES & SAFETY

04 原子七类 · 规则的最小构件

PART 2 · SEVEN ATOMS

原子(atom)是规则的砖。XML 具体语法里它们是 `swrlx:` 元素,Protégé 里显示的就是这些名字。

原子	写法	含义
类原子 `classAtom`	`C(?x)`	?x 是类 C 的实例
数据范围原子 `datarangeAtom`	`D(?x)`	?x 落在数据范围 D 内(整型、字符串等)
个体属性原子 `individualPropertyAtom`	`P(?x, ?y)`	对象属性 P 把个体 ?x 连到个体 ?y
数据属性原子 `datavaluedPropertyAtom`	`D(?x, lit)`	数据属性 D 把 ?x 连到一个字面量
同一原子 `sameIndividualAtom`	`sameAs(?x,?y)`	?x 与 ?y 是同一个个体的两种叫法
不同原子 `differentIndividualsAtom`	`differentFrom(?x,?y)`	?x 与 ?y 是两个不同的个体
内置原子 `builtinAtom`	`builtIn(r, …)`	内置关系 r 对参数成立(比较、算术等)

TWO NOTES

① 规范明说 `sameAs` / `differentFrom` 是**语法糖**(syntactic sugar),不增加表达力,只是让规则好读。

② **内置谓词**(built-in)挂在 `swrlb:` 前缀下(下一页全表)。规范对元数很松——不要求固定 arity,元数给错不算语法错误,只是该原子「不可满足」。

七类原子 · 抽象写法一网打尽

```
:成人(?p)                          # 类原子:年龄属性推断出的实例也认
:及格分数(?g)                      # 数据范围原子:?g 落在自定义数据范围
:选修了(?s, ?c)                    # 个体属性原子:人 连 课程
:年龄(?p, ?n)                      # 数据属性原子:人 连 数值
sameAs(?a, ?b)                     # 同一原子:语法糖
differentFrom(?a, ?b)              # 不同原子:语法糖,却常是规则关键前提
swrlb:greaterThan(?n, 18)          # 内置原子:比较发生在推理解算时
```

变量只有 `?x` 一种写法

变量是规则的灵魂:同一变量在同一条规则内处处同值,跨规则无记忆。要「或」怎么办?拆成多条规则——SWRL 的 head 是合取,不是析取。

05 变量与安全条件

PART 2 · VARIABLES & SAFETY

SAFETY CONDITION · 规范原文

"only variables that occur in the antecedent of a rule may occur in the consequent"

—— 只有出现在**前提**里的变量,才允许出现在**结论**里。

为什么需要 WHY

- 结论里冒出前提没提的变量,机器不知道绑给谁
- 是凭空造新个体,还是对所有人都成立?语义失控
- 安全条件把规则钉死在「从已知推新知」

代价有多大 NOT MUCH

- 规范论证:安全条件**不损失表达力**
- 「存在新个体」可由 OWL 的 someValuesFrom 表达
- 不满足的规则,通行叫法:不安全规则(unsafe)

06 内置谓词 · swrlb: 七大门派

PART 3 · BUILT-INS

命名空间 `http://www.w3.org/2003/11/swrlb`,大量借用 XQuery/XPath 函数库。

门派	代表
比较	`equal notEqual lessThan lessThanOrEqual greaterThan greaterThanOrEqual`
数学	`add subtract multiply divide integerDivide mod pow abs ceiling floor round sin cos tan`
布尔	`booleanNot`
字符串	`stringConcat stringEqualIgnoreCase substring stringLength upperCase lowerCase contains startsWith matches replace tokenize`
日期时间	`dateTime date time yearMonthDuration addDayTimeDurations subtractDates`
URI	`resolveURI anyURI`
列表	`listConcat listIntersection listSubtraction member length first rest sublist empty`

列表门派的坑

规范明说:列表类 built-in 在 OWL DL / OWL Lite 里**不可用**——RDF 式列表只有在 OWL Full 里才被当成个体对待。

内置组合实战 · 字符串 + 数值

```
# 邮箱在 example.org 域、且工号大于 1000 的,是资深员工
:员工(?e) ∧ :邮箱(?e,?m) ∧ :工号(?e,?no)
  ∧ swrlb:contains(?m, "@example.org")
  ∧ swrlb:greaterThan(?no, 1000)
  → :资深员工(?e)
```

内建的「方向性」

built-in 是**关系判断**不是赋值语句——`swrlb:add(?z,?x,?y)` 在 ?z 未绑定时可当「求和」用,三个都已知时就是「验算」。能不能反推,取决于内置的定义与推理机实现。

07 属性原子只有二元 · 多条件怎么写

PART 3 · BINARY ONLY

SWRL 属性原子只能连两项,`P(?x, ?y)`,没有三元、四元。碰上「谁向谁付了多少」这种三段式,得先把事件本身变成一个个体(具体化,reification),再往它身上挂二元属性。

```
# 把「这笔支付」建成个体 ?pay,挂三个二元属性,再接数值比较
:支付记录(?pay) ∧ :付款方(?pay,?c) ∧ :收款方(?pay,?p) ∧ :金额(?pay,?amt)
  ∧ swrlb:greaterThan(?amt, 10000)  →  :需披露(?c, ?p)
```

别拿规则当子类用

`Student(?x) ⇒ Person(?x)` 这类规则与 `SubClassOf(Student Person)` 等价——留给 OWL 推理机,别浪费 SWRL。规则真正的主场是「要跨属性组合、要比大小、要拼字符串」的情形。

08 可判定性 · SWRL 最大的代价

PART 3 · DECIDABILITY

规范原文

"OWL DL becomes undecidable when extended in this way as rules can be used to simulate role value maps."
—— 这样一扩展,OWL DL 就**不可判定**了:规则能拿来模拟「角色值映射」(role value map)。

问题 UNDECIDABLE

- OWL DL 本来可判定:推理一定跑得完、给得出确定答案
- 加上规则就失守:理论上可能永远算不完
- 根子在于规则表达力太强,能搭出已知会破坏可判定性的构造

对策 DL-SAFE

- 变量只能绑定到**显式命名的个体**(named individual)
- 不许绑定推理过程中凭空生成的匿名个体
- 切断「本体造个体 → 规则再推」的循环,恢复可判定

实践含义

这条限制叫 **DL-safe rules**(DL 安全规则),由 Motik 等人在 2005 年提出,正是为了把 SWRL 拉回可判定的地面。主流推理机实现的基本都是这个子集,而不是 SWRL 全集——你写的规则能不能跑,取决于推理机认不认。

09 分工 · OWL / 链 / SWRL / SPARQL

PART 4 · DIVISION OF LABOR

工具	干什么	产生新知识?
OWL 公理	描述静态结构(类、属性、限制)	**会 · 推理机演绎**
OWL 2 属性链	「连走两步推出第三条」的规则形状	**会 · 稳在可判定区间**
SWRL	带内置函数的任意条件规则	**会 · 需 DL-safe 护栏**
SPARQL	查询检索(CONSTRUCT/INSERT 除外)	**不会 · 只读为主**

一句话

能用属性链写的(PL.04 §08),优先属性链;真正落到 SWRL 的是链写不出的那类:要内置函数、要判断「两个个体不同」、要跨多个属性组合的条件规则。

10 推理机支持度 · 谁真的在跑 SWRL

PART 4 · ENGINE SUPPORT

工具	路线
Pellet / Openllet	Tableau;支持 DL-safe SWRL 及内置谓词
Protégé SWRLTab	背后是 Drools 规则引擎
HermiT	Tableau;SWRL 支持有限
Jena Rules / RDFox	自有规则语法(Datalog 谱系),**不是** SWRL

碎

同一条规则,换个引擎行为可能就不一样——这是 SWRL 论文里常见、工程里少用的原因之一。写之前先锁定目标引擎。

11 实操坑清单

PART 4 · PITFALLS

01 身份先记住:Member Submission 不是 W3C 标准——项目文档里别说「W3C 推荐的 SWRL」。

02 能写成属性链的,别用 SWRL——保住可判定性,也保住工具兼容。

03 结论里不许出现前提没绑定的变量——安全条件,越界即规则作废。

04 没有「否定即失败」:写不出「如果不知道 P,则…」这种话。

05 规则是单调的:只增不减,推出来的结论撤不回去。

06 小心规则爆炸:不谨慎的规则配上开放世界,能推出海量结论。

07 没有优先级、没有异常处理——不像业务规则引擎那样能排先后。

08 swrlb 列表类内置在 DL/Lite 里不可用,用了就是 Full。

09 换推理机可能换结果:DL-safe 支持度参差,上线前务必实测。

10 规则标注与 URI 不携带语义——别指望给规则挂 meta 就能影响推理。